WEEK 2 · DAY 3, PART 2 · OVERVIEW

pandas

Combining and Loading, Part 2



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 5 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Thirteenth Note

Part 1's averages gave every bond one vote. Give me spread and duration weighted by market value, by sector too, and tell me which sector moves and why. Then market value by sector and grade in one grid. How much matures in the next three years? Send it as a workbook I can open, and leave nothing behind.

ABOUT 50 MIN WITH EXERCISES / 5 SECTIONS / 5 EXERCISES

Your Route Through Day 3, Part 2

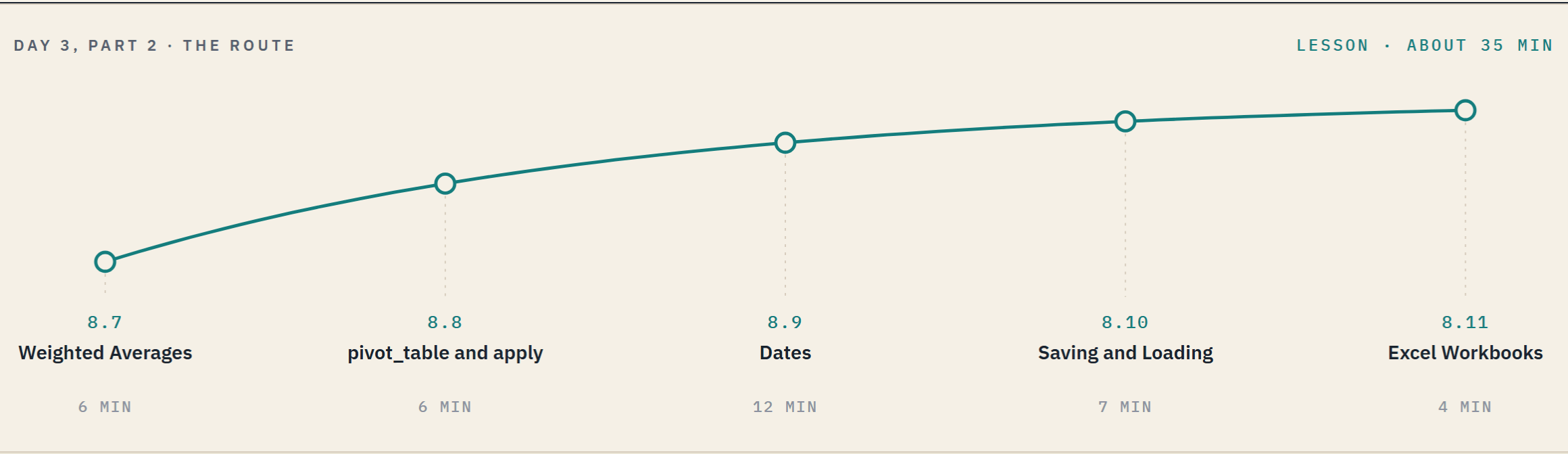


FIGURE 1 Five sections, in order, with about how long each takes.

Time. About 35 minutes for the lesson, then about 15 for the five exercises.

8.7 Weighted Averages

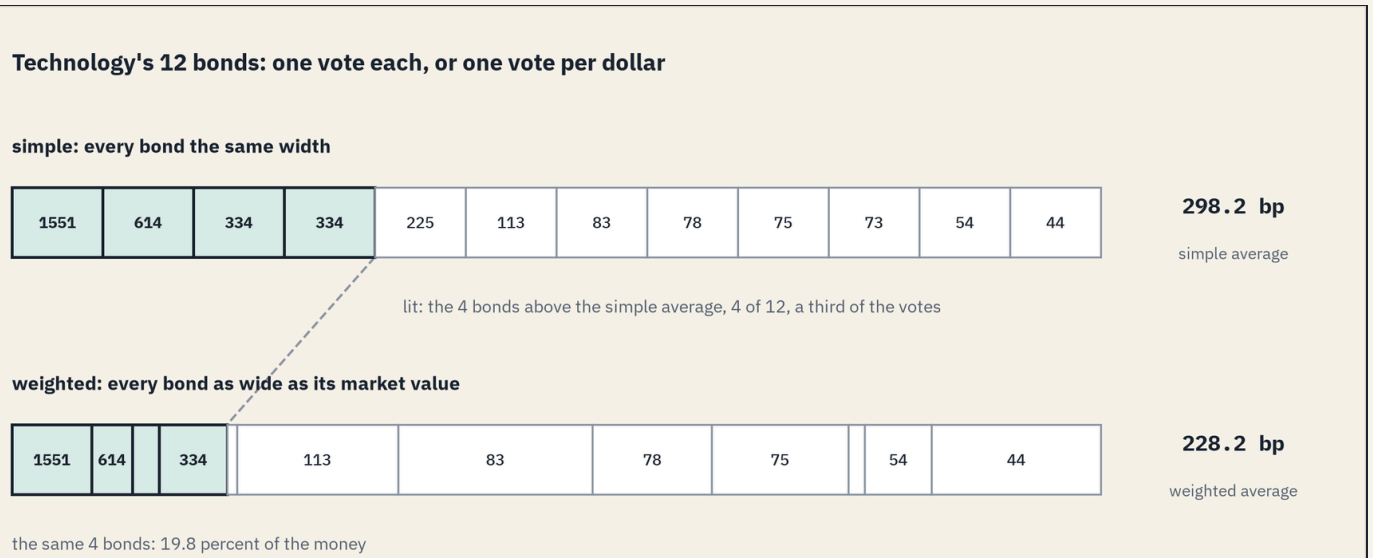


FIGURE 2 Technology's 12 bonds: counted equally, the average is 298.2 bp; sized by market value, it is 228.2 bp.

WEIGHTED BESIDE SIMPLE, BY SECTOR

SECTOR	SIMPLE	WEIGHTED	GAP
Consumer	218.9	209.4	-9.5
Energy	171.6	176.3	4.8
Financials	172.3	176.1	3.8
Health Care	141.8	128.0	-13.7
Industrials	220.6	234.0	13.4
Materials	182.4	180.7	-1.7
Technology	298.2	228.2	-69.9

Neither is wrong: a typical bond against a typical dollar.

One Vote per Dollar

THE LONG WAY

```
oas_x_value = book['oas'] * book['market_value']
weighted_oas = oas_x_value.sum() / book['market_value'].sum()
print('Simple:', round(book['oas'].mean(), 2), 'Weighted:', round(weighted_oas, 2))
```

```
Simple: 185.75 Weighted: 180.46
```

DURATION, THE SAME TWO STEPS

```
duration_x_value = book['duration'] * book['market_value']
weighted_duration = duration_x_value.sum() / book['market_value'].sum()
print('Simple:', round(book['duration'].mean(), 2), 'Weighted:', round(weighted_duration, 2))
```

```
Simple: 6.47 Weighted: 6.25
```

EXCEL SIDE BY SIDE

NumPy Part 1's `=SUMPRODUCT(M2:M201, S2:S201) / SUM(S2:S201)` weights the whole book, OAS in M and market value in S.

INTO THE SECTOR SUMMARY

SECTOR	BONDS	AVG_OAS
Consumer	26	218.88
Energy	17	171.59
Financials	26	172.31

Weight by market value, and spread and duration both come in lower.

8.8 pivot_table and apply

pd.pivot_table: long rows in, a grid out

book, 4 of 200 rows

sector	grade	market_value
Financials	Investment Grade	5435500.0
Industrials	High Yield	253087.5
Transportation	High Yield	1505940.0
Transportation	High Yield	475850.0

...

`columns='grade'`

Columns: one per grade, across the top

sector	High Yield	Investment Grade	All
Consumer	19,016,165.0	37,613,432.5	56,629,597.5
Energy	7,347,315.0	31,482,690.0	38,830,005.0
...			
Utilities	5,335,625.0	23,670,837.5	29,006,462.5
All	169,901,850.0	320,917,365.0	490,819,215.0

`index='sector'` Rows: down the side

`values='market_value', aggfunc='sum'` Values: the sum in each cell

`margins=True` Grand Totals: the green All row and All column

FIGURE 3 Sector down the side, grade across the top, market value in the cells, and margins adding the All totals.

A LABEL FROM A RULE

```
book['price_label'] = book['price'].apply(price_label)
book['price_label'].value_counts()
```

```
price_label
Discount    113
Premium      87
Name: count, dtype: int64
```

COUPON BY LABEL

```
book.groupby('price_label')['coupon'].mean().round(2)
```

```
price_label
Discount    5.36
Premium     6.69
Name: coupon, dtype: float64
```

One line, one PivotTable, totals included.

8.9 Dates

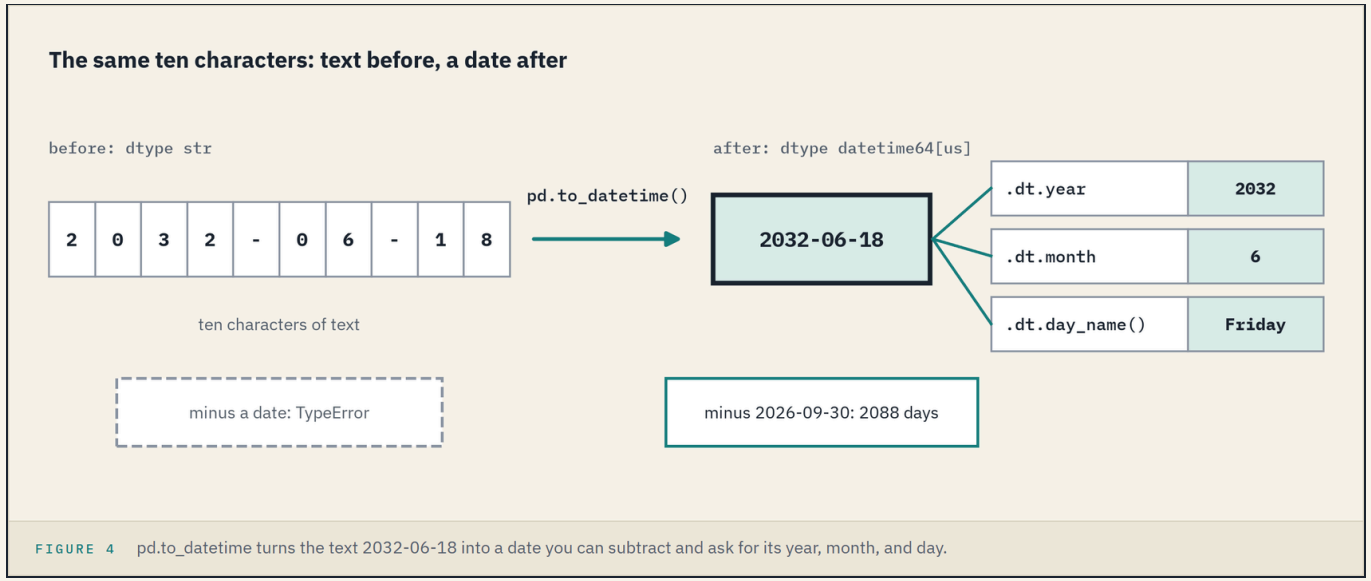


FIGURE 4 pd.to_datetime turns the text 2032-06-18 into a date you can subtract and ask for its year, month, and day.

PULL THE PARTS OUT

```
book['maturity_year'] = book['maturity'].dt.year
book['maturity_month'] = book['maturity'].dt.month
book['maturity_day'] = book['maturity'].dt.day_name()
```

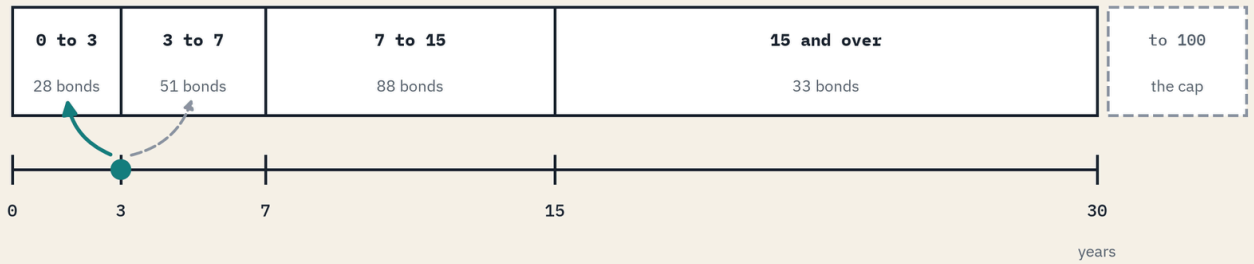
WHAT CAME OUT

	TICKER	MATURITY
0	BEZE	2032-06-18
1	BEZH	2029-04-17
2	BEZL	2030-06-03

Text cannot be subtracted: convert it first.

Maturity Buckets

pd.cut: four ranges, each one takes its upper edge

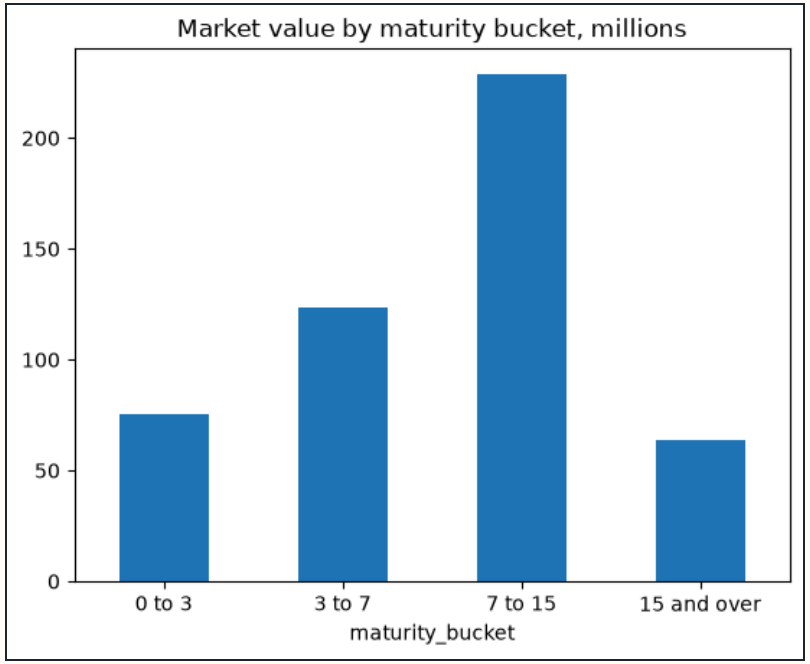


A bond at exactly 3.0 years: pd.cut puts it in 0 to 3 (green arrow).

VLOOKUP approximate on the edges, or pd.cut(..., right=False), puts it in 3 to 7 (dashed).

FIGURE 5 Maturity buckets with edges at 3, 7 and 15 years; a bond at exactly 3.0 years falls in 0 to 3.

MARKET VALUE BY BUCKET



28 bonds and 75.1 million inside three years; the most money 7 to 15 years out.

8.10 Saving and Loading

SAVE AND READ BACK

```
book.to_csv('desk_book_with_grades.csv')
read_back = pd.read_csv('desk_book_with_grades.csv')
print('Saved:', book.shape, 'Read back:', read_back.shape)
```

```
Saved: (200, 30) Read back: (200, 31)
```

WITHOUT THE ROW LABELS

```
book.to_csv('desk_book_with_grades.csv', index=False)
read_back = pd.read_csv('desk_book_with_grades.csv')
print('Saved:', book.shape, 'Read back:', read_back.shape)
```

```
Saved: (200, 30) Read back: (200, 30)
```

A CSV keeps values, not types: name the identifiers and the dates when reading it.

8.11 Excel Workbooks

What comes back from a file

what was saved	CSV	workbook (.xlsx)
row labels (the index)	written unless index=False; back as a column	written unless index=False; back as a column
a digit-only CUSIP	read as a number, zeros gone: dtype=	stored as text, read as a number: dtype=
dates	read as text: parse_dates=	**dates, as saved**
bucket order (category)	plain text, the order lost	plain text, the order lost

green: back as saved; white: right only with the keyword named; dashed: gone for good

FIGURE 6 What comes back from a CSV and from a workbook: row labels, a digit-only CUSIP, dates, and bucket order.

WHAT SURVIVED THE TRIP

	SAVED	READ BACK
cusip	str	str
maturity	datetime64[us]	datetime64[us]
maturity_bucket	category	str

ONE WORKBOOK, TWO SHEETS

```
for name, table in all_sheets.items():
    print(name, table.shape)
```

```
book (200, 30)
by_sector (10, 6)
```

A few lines write the report; next month they run again.

You Can Now

- Report a market-value-weighted OAS or duration, by sector too, and say why it differs from the simple one.
- Join two tables on their row labels, and lay two groupings out as a pivot table.
- Label bonds by a rule with **`apply`**, **`lambda`**, or **`np.where`**.
- Turn date text into dates and build a maturity profile: years left, buckets, market value by bucket.
- Save a table to a CSV and read it back with no stray index, identifiers as text, and dates as dates.
- Write a report to a two-sheet Excel workbook and read it back.

NEXT
WEEK 2 · DAY 4

Case Study: A First Look at the Data

Three tables arrive from three systems, and each one gets a careful look before anything is joined.